Sukio: A Marketplace for Machine Learning Solutions

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Keshav Juneja, CSED* Satwik Sinha, CSED†

August 19, 2026

1 Higher-order goal

...secondary framing

Sukio aims to make practical machine learning capability more accessible. People with a data problem should be able to obtain a credible starting solution without first becoming ML specialists, while ML engineers should have a structured way to package, demonstrate, and monetise their work. The immediate engineering objective is a deployable marketplace that helps users discover, evaluate, and acquire suitable ML solutions.

2 Time-to-value

...primary heuristic

- The first usable increment will let a buyer browse verified listings, inspect compatibility and performance details, and request a solution.
- Weekly iterations will add the Build, Buy, and Hire journey in order, producing testable user value early.
- CI will run linting, unit tests, and integration tests on each change; a staging deployment will support rapid feedback from student users and ML practitioners.

3 Problem Statement

An individual or small team with a machine-learning problem often faces three costly alternatives: build and tune a model from scratch, search through repositories with unclear suitability, or find and contract an expert manually. Existing repositories make discovery possible but rarely provide consistent compatibility information, trustworthy performance evidence, a safe commercial transaction, or an escalation path for specialised work.

ML engineers face the complementary problem: a model, pipeline, or training recipe they have already built is difficult to package, demonstrate, license, and sell repeatedly. Consequently, useful ML assets remain scattered and buyers cannot confidently assess whether an asset will work with their data and hardware.

Contextual relevance: student developers, startups, and local organisations increasingly need ML-enabled features but may not have dedicated ML expertise. A focused marketplace can reduce the time from problem identification to a usable baseline or expert engagement.

*Roll No: 1024030692
†Roll No: 1024030699

4 Proposed Solution *...Software Proposition*

4.1 Overview

Sukio is a web marketplace for ML models, pipelines, recipes, utilities, and expert services. Its experience is organised as a three-level funnel:

1. **Build:** an AutoML baseline helps a user understand their dataset and obtain an initial result.
2. **Buy:** the marketplace recommends compatible, reusable ML assets when a stronger or faster solution may already exist.
3. **Hire:** users can submit a scoped request to an ML expert when no existing listing meets their need.

AutoML is therefore a discovery and baseline engine, rather than the product's sole identity. It creates useful evidence—dataset characteristics, problem type, and baseline metrics—that can guide marketplace recommendations.

4.2 Marketplace listings

Each listing will present a clear product card and detail page containing:

- the asset type and description (for example, a classification model, preprocessing pipeline, or fine-tuning recipe);
- input and dataset requirements, supported frameworks, hardware requirements, and version compatibility;
- performance metrics appropriate to the task, such as F1, AUROC, RMSE, latency, and memory/GPU requirements;
- a sample-data demo, price model, and license terms; and
- seller profile, ratings, downloads, and a verification status.

The prototype will prioritise models and complete pipelines. The data model will support future categories including evaluation packages, ML utilities, reproducible recipes, legally distributable datasets, and expert services.

4.3 Core workflow

1. A buyer enters a problem description and optionally uploads a small tabular dataset or its metadata.
2. Sukio identifies the problem type and runs a constrained baseline workflow for supported datasets.
3. The buyer sees the baseline result and selects *Find a Better Model*.
4. Sukio ranks listings using task type, feature and target characteristics, sample size, baseline metric, and hardware constraints.
5. The buyer compares verified listings, tests a demo where available, and creates a purchase/request record.
6. If no listing is suitable, the buyer posts a scoped requirement for an expert; an expert can respond with an offer.

5 Solution Approach *…Engineering focus*

5.1 Verification and trust

Sukio will distinguish seller-reported figures from platform-verified results. During publication, a seller supplies model metadata, a test interface, supported input schema, and declared environment. For the prototype, an administrator-controlled verification run executes a curated test case and records reproducible metrics, latency, and runtime compatibility. A *Sukio Verified* badge is issued only after that run succeeds.

Buyers may also submit anonymised outcome feedback. Aggregated community results will be displayed separately from platform verification so ratings communicate both user experience and observed performance. Listing publication will require a rights declaration and licence selection; moderation prevents obviously incomplete or misleading listings.

5.2 Recommendation and AutoML scope

The first version limits AutoML to small, structured classification/regression datasets. It will compare a small set of established baseline algorithms and record the best validation metric. Recommendation is intentionally transparent and heuristic-based: it filters incompatible listings and scores the remaining listings by problem type, schema match, resource limits, and verified performance. This is an engineering integration task, not a claim of novel ML research.

5.3 Web architecture

The system follows a modular web architecture:

- **Frontend:** responsive buyer, seller, and administrator interfaces for search, comparison, publishing, and requests.
- **Backend API:** authentication and roles, listing lifecycle, search/recommendation, purchase/request records, reviews, and audit events.
- **Worker service:** bounded AutoML runs and verification jobs, executed asynchronously with resource limits.
- **Database and object storage:** relational records for users, listings, licences, metrics, and transactions; object storage for approved artifacts and demo assets.

5.4 CI/CD and fast delivery enabler

CI/CD will automatically build the application, run API and UI tests, validate migrations, and publish a staging build. Each workflow increment will be separately testable: listing discovery first, seller publication and verification second, then AutoML-led recommendation and expert requests.

6 Evaluation Criterion *…measurable and attributable*

6.1 Primary evaluation metric

Time-to-shortlist (TTS): elapsed time from a buyer submitting a supported problem description/dataset profile to receiving at least three compatible marketplace recommendations or an explicit "no compatible listing" result.

- Target: median TTS $\leq$ 60 seconds for the prototype catalogue and supported tabular datasets.

- Attribution: measured using server-side timestamps from submission through recommendation response.

6.2 Secondary evaluation metrics

- **Recommendation relevance:** at least 80% of pilot users rate one or more shortlisted listings as relevant (4/5 or higher).
- **Verification coverage:** 100% of listings shown in recommended results display either a verified metric record or an explicit unverified status.
- **Publication completeness:** at least 90% of submitted prototype listings pass required metadata, licence, and compatibility validation without manual rework.
- **Workflow reliability:** at least 95% of supported baseline/verification jobs complete with a recorded result during pilot testing.

6.3 Pilot validation plan

- Seed a controlled catalogue of 10–15 representative model/pipeline listings with varied schemas and resource requirements.
- Recruit 10–15 student buyers and 3–5 student ML-engineer sellers for task-based testing.
- Ask buyers to locate or request a solution for defined datasets; collect timestamps, system logs, and short relevance feedback.
- Run verification against controlled test inputs and compare stored results with expected outcomes.

7 Scalability *...with foresight*

1. **Scalable theoretically:** listing search will use indexed metadata and paginated results; recommendation and verification are isolated from the interactive API through a job queue; stateless API instances can scale independently.
2. **Operationally deployable:** the web application, worker service, managed database, and object storage can be containerised and deployed on standard cloud platforms. Compute quotas, artifact size limits, and asynchronous processing bound the cost and safety of ML workloads.

8 Engine availability heuristic

The project relies on mature components rather than unproven research infrastructure:

- web frameworks, REST APIs, authentication, and managed relational databases;
- established AutoML/baseline libraries for structured data;
- container execution or managed job queues for bounded verification runs;
- object storage and virus/file-type validation for artifacts;
- test frameworks, CI runners, and staging deployment services.

9 Project scope and deliverables *…iteration-friendly*

9.1 Initial deliverable *…first iteration*

- buyer-facing catalogue, filters, search, and listing detail pages;
- seller listing submission with required metadata, compatibility, price, and licence fields;
- administrator review and a recorded prototype verification result;
- authenticated roles and a purchase/request intent record (no live payment processing);
- CI for linting, unit tests, API tests, and staging deployment.

9.2 Subsequent deliverables

- constrained tabular AutoML baseline and the *Find a Better Model* recommendation flow;
- asynchronous verification jobs, verification badges, and seller-versus-verified metric display;
- ratings and anonymised outcome feedback;
- expert request board and seller responses;
- refined ranking, audit logs, and administrator moderation tools.

10 Risks and mitigations

- **Unsafe or unlicensed artifacts:** restrict prototype uploads, scan files, require rights declarations, use isolated verification, and do not execute arbitrary uploaded code in the web process.
- **Misleading performance claims:** label seller-reported and verified metrics separately; use controlled test inputs and retain verification logs.
- **Compute cost and long-running jobs:** limit dataset size, supported task types, execution time, and concurrency; process jobs asynchronously.
- **Marketplace cold start:** seed a controlled catalogue and run a pilot with student sellers before relying on external supply.
- **Payment and legal complexity:** represent transactions as intent records in the course prototype; defer payment settlement and formal licence enforcement to future work.

11 Summary

Sukio is proposed as a marketplace where users can build, buy, and commission machine-learning solutions. A constrained AutoML baseline helps users discover their needs; verified, compatible marketplace listings offer reusable solutions; and expert requests cover specialised gaps. The initial scope delivers marketplace trust and discovery quickly, while later iterations add recommendation and expert workflows. Its success will be measured by the speed and perceived relevance of compatible shortlists, transparent verification coverage, and reliable completion of supported jobs.